

The chosen domain was film production, the system in particular is built for general film production tasks with no particular focus on filming or editing. The system is scalable and follows the open closed principle which makes the code maintainable.

[illegible]

GoF Pattern	Participant	Class
Composite	Component	ToDoItem
	Leaf	ToDoTask
	Composite	ProductionGroup
Decorator	Component	ToDoItem
	ConcreteComponent	ToDoTask, ProductionGroup
	Decorator (abstract)	TaskDecorator
	ConcreteDecorator	PriorityDecorator, CostAdjustmentDecorator
State	Context	ToDoTask
	State (abstract)	State
	ConcreteState	Planning, Progressing, Paused, Cancelled, Finished
Iterator	Aggregate (factory method)	ToDoItem::createIterator(type)
	Iterator (abstract)	ItemIterator
	ConcreteIterator	standardTraverseIterator, EstimatedCostIterator

Relationship specifics

Every class has a virtual destructor

Production group owns its children

TaskDecorator owns the object it points to (it can only point to one other object)

ToDoTask has ownership its own state object (it can only point to one State object type at a time)

Inheritance relationships:

- Cancelled, Finished, Paused, Planning, Progressing Inherit from the superclass State
- ToDoTask, ProductionGroup, TaskDecorator inherit from the superclass ToDoItem
- PriorityDecorator, CostAdjustmentDecorator inherit from the superclass TaskDecorator
- StandardTraversallterator, EstimatedCostlterator inherit from the superclass Itemlterator

Rationale:

The ownership policy we implemented is as follows : (ProductionGroup owns its children, TaskDecorator owns the object it points to, ToDoTask owns its state) these ownership choices are made to simplify the memory management process from the perspective of the client, it decouples the client from the classes it uses.

We have opted for a parameterized createliterator() operation, this is due to the fact that we don't necessarily have two completely different data structures for the iterators to iterate over. Instead we have different ways of iterating over the Composite structure. It takes in a string parameter and expects it to either be "STANDARD" or "COST" and returns either a StandardTraversallterator, EstimatedCostlterator respectively.

The iterator is owned by the Client that uses it, this is the most straight forward choice and makes the most sense.

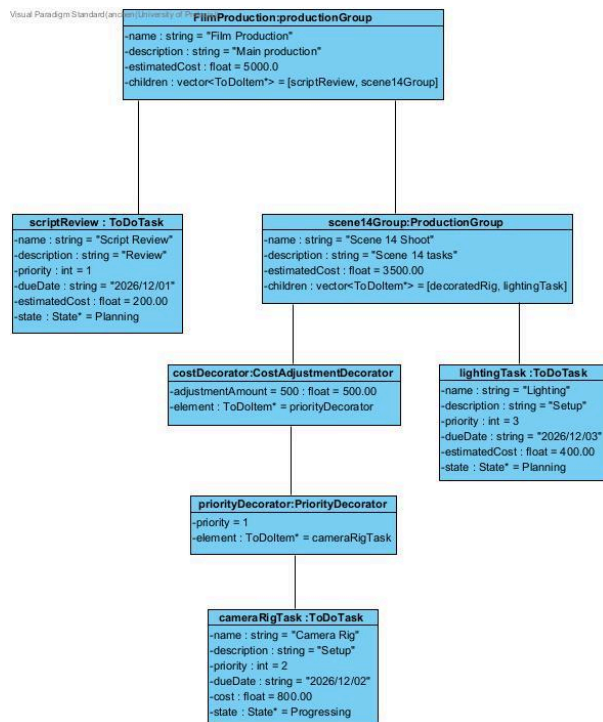
Task 3:

Scenario 1 — Morning Production Meeting: Traverses the full shoot-day hierarchy two ways — structural order and cost order — proving both iterators coexist independently. A late task is added to Scene 14 mid-traversal; an already-open iterator is shown *not* to see it (snapshot policy), while a fresh iterator afterward does.

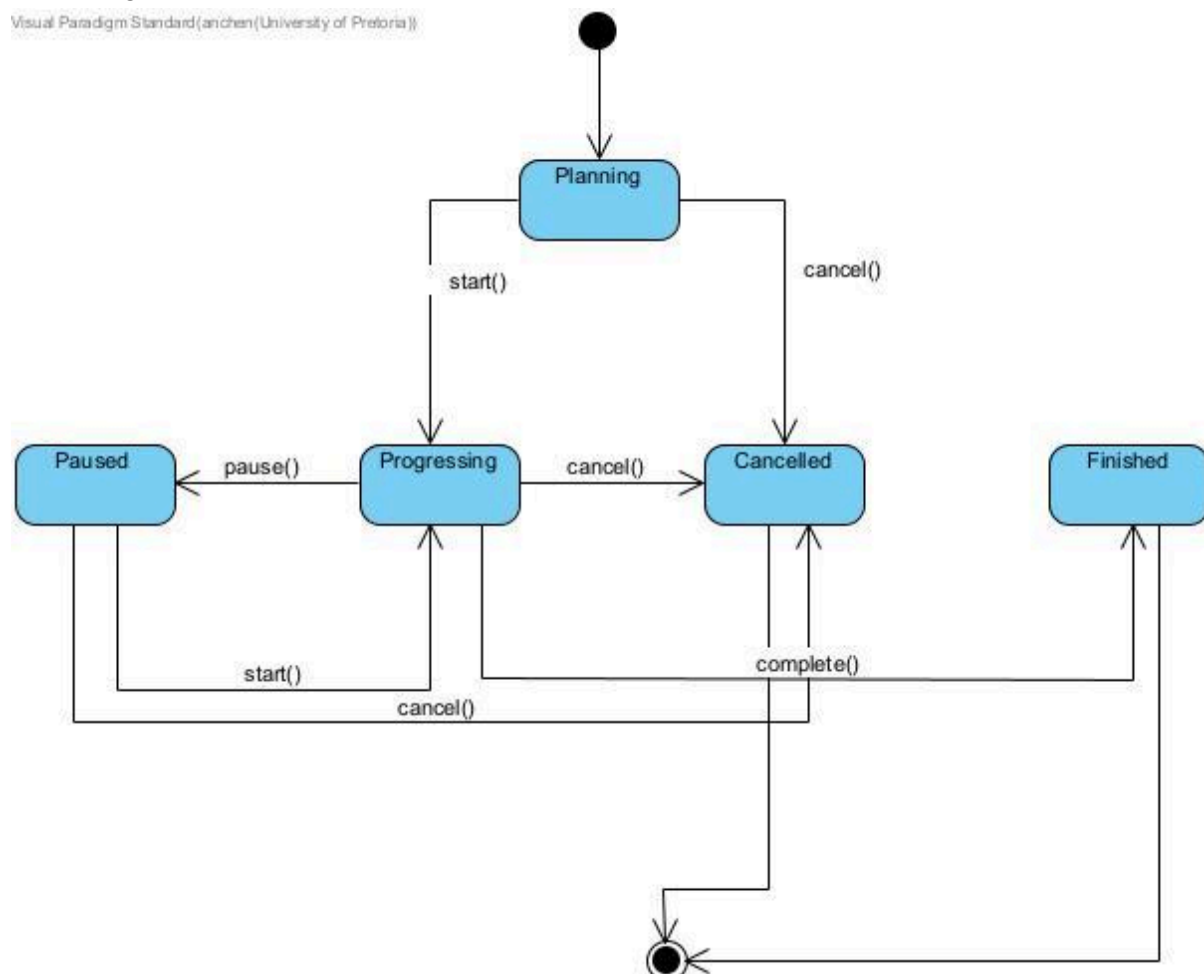
Scenario 2 — Stunt Task Scheduling Problem: Follows one task through its full lifecycle, including an invalid transition (completing while paused). It's wrapped in two stacked decorators (priority, then cost) and used only through the base ToDoItem interface. It's later moved between groups — the decorated object, not the raw task, is what gets relocated, preserving single ownership.

Task 4:

Object Diagram:



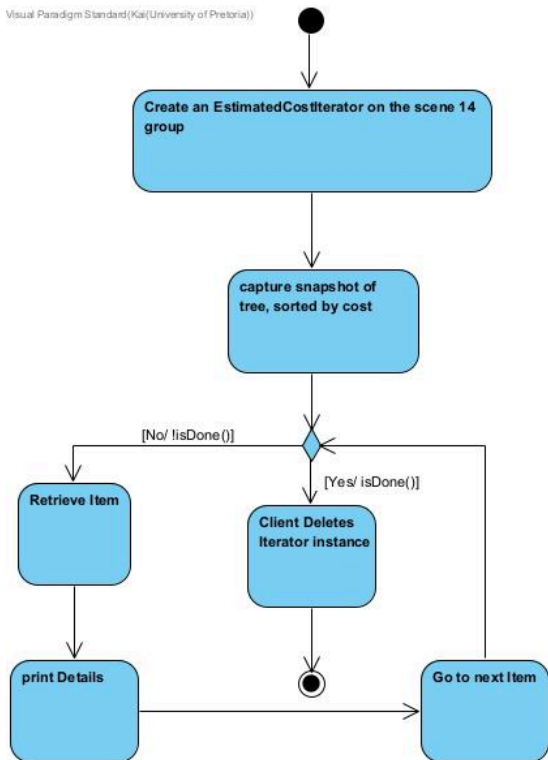
State diagram:



Activity Diagrams:

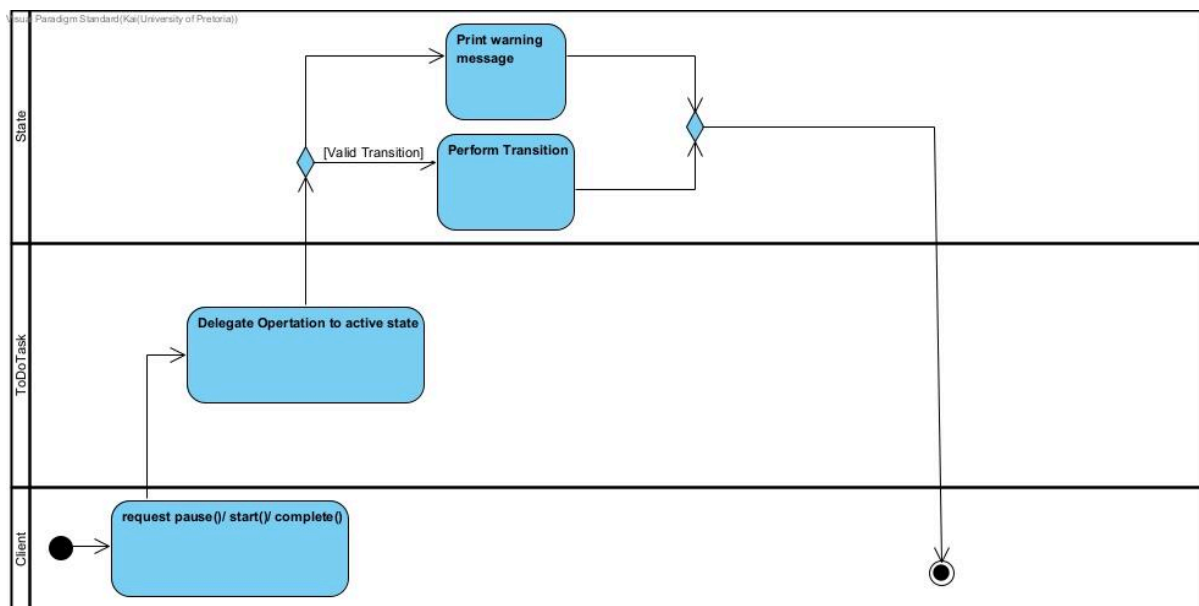
1.

Visual Paradigm Standard (Kai University of Pretoria)



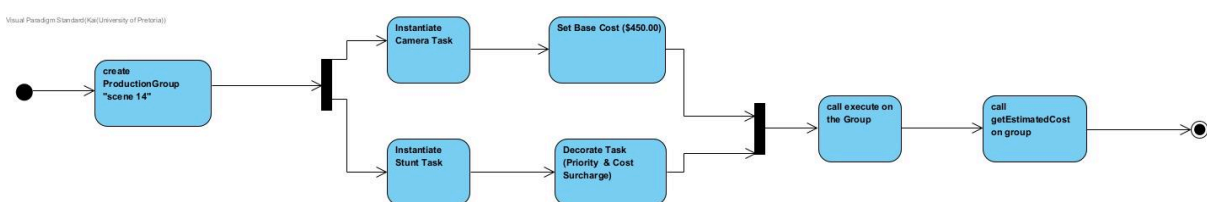
2.

Visual Paradigm Standard (Kai University of Pretoria)



3.

Visual Paradigm Standard (Kai University of Pretoria)



Task 5

During debugging, a segmentation fault was encountered. We attempted to test the method `start()` in a `Paused` state:

```
int main()
{
    State* state = new Paused();
    ToDoTask todo("Catering", "Set up catering", state);
    todo.start();
}
```

Which leads us to the following gdb session:

```
(gdb) s
ToDoTask::setState (this=0x7fffffff300, state=0x555555786e0) at ToDoTask.cpp:77
77      delete state;
(gdb) s
Progressing::~Progressing (this=0x555555786e0, __in_chrg=<optimized out>) at /home/henry/gitted/COS214/Prac4/State.h:46
46      class Progressing : public State {
(gdb) s
Progressing::~Progressing (this=0x555555786e0, __in_chrg=<optimized out>) at /home/henry/gitted/COS214/Prac4/State.h:46
46      class Progressing : public State {
(gdb) s
State::~State (this=0x555555786e0, __in_chrg=<optimized out>) at /home/henry/gitted/COS214/Prac4/State.h:15
15      virtual ~State() = default;
(gdb) s
ToDoTask::setState (this=0x7fffffff300, state=0x555555786e0) at ToDoTask.cpp:78
78      if (!state)
(gdb) s
81      this->state = state;
(gdb) print state
$2 = (State *) 0x555555786e0
```

Notice first that 'state' (at 0x555555786e0) is deleted, but `this->state` is then assigned to 'state', so that when we run `todo.start()`, which in turn runs `this->state->start(this)`, we obtain a SIGSEGV.

We have this:

```
void ToDoTask::setState(State* state)
{
    delete state;
    if (!state)
        throw "state cannot be null";
    else
        this->state = state;
}
```

Which must be changed to this:

```
void ToDoTask::setState(State* state)
{
    delete this->state;
    if (!state)
        throw "state cannot be null";
    else
        this->state = state;
}
```

We have the following valgrind output for our demo main:

```
==774907==  
==774907== HEAP SUMMARY:  
==774907==      in use at exit: 0 bytes in 0 blocks  
==774907==    total heap usage: 127 allocs, 127 frees, 85,341 bytes allocated  
==774907==  
==774907== All heap blocks were freed -- no leaks are possible  
==774907==  
==774907== For lists of detected and suppressed errors, rerun with: -s  
==774907== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Task 6

We divided the work mostly by tasks and by patterns: each pattern in the design was assigned to a team member, which allows for easy integration of code since we each worked on our own classes.

For git, we followed the strategy of creating feature branches and subsequent pull requests. The below image shows, for example, that Heinrich created a feature branch 'TransitionRules' to update the way transitions occur, Kai worked on the Iterator header and implementation, and Anchen worked on the composite pattern represented by the 'ProductionGroup' class.

☐	 Added dockerfile #15 by u25030932 was merged 3 hours ago
☐	 Finished first two diagrams of Task 4 #14 by anchenk was merged 10 hours ago
☐	 added the Iterator .cpp and .h #13 by Kaizo344 was merged 4 hours ago
☐	 Transition rules #12 by u25030932 was merged 16 hours ago
☐	 Got rid of const string& and added decorators #11 by u25030932 was merged 16 hours ago
☐	 readme #10 by anchenk was merged 20 hours ago
☐	 Basics of the readme #9 by anchenk was merged 20 hours ago
☐	 Basics of the readme #8 by anchenk was merged 20 hours ago
☐	 Added states #7 by u25030932 was merged 20 hours ago
☐	 Finished Productio.cpp #6 by anchenk was merged 20 hours ago
☐	 Implementations #5 by anchenk was merged yesterday
☐	 ToDoTask implemented partially: getIterator unimplemented #4 by u25030932 was merged yesterday
☐	 Finished productionGroup.h #3 by anchenk was merged yesterday
☐	 ToDoTask.h finished #2 by anchenk was merged yesterday
☐	 ToDoItem.h file completed #1 by anchenk was merged yesterday